Aror University of Art, Architecture, Design & Heritage Sukkur.

Department of Artificial Intelligence

Data Structures (Fall-2026)

LAB No. 3

Objective of Lab No. 3:
After performing lab1, students will be able to:

- Understand the basic representation and operations of arrays using Python lists.
- Implement array traversal, insertion, deletion, updating, and linear searching.
- Understand how index access and element shifting affect algorithm performance.
- Measure execution time using time.perf_counter().
- Analyze time and auxiliary space complexity using Big-O notation.

Note: Python lists are used as array-like structures for this lab. Although Python lists are dynamically sized, the exercises focus on indexed access, traversal, shifting, updating, and linear searching—the core ideas required for array analysis.

Lab Tasks:

Task 01 — Simple Array (Python List)

Create and display a simple array of student data. Access elements using known indexes and observe constant-time indexed access.

Solution:

```
import time
marks = [75, 82, 91, 68, 88]
start_time = time.perf_counter()
print("Array:", marks)
print("First element:", marks[0])
print("Third element:", marks[2])
print("Last element:", marks[-1])
end_time = time.perf_counter()
print("Length:", len(marks))
print("Execution Time:", end_time - start_time, "seconds")
```

```
print("Time Complexity: O(n) for printing all indexes")
print("Auxiliary Space Complexity: O(1)")
```

Task 02 – Access Known Positions

Solution:

```
import time

student_ids = [101, 102, 103, 104, 105, 106, 107, 108]
start_time = time.perf_counter()
print("Index 1:", student_ids[1])
print("Index 4:", student_ids[4])
print("Index 7:", student_ids[7])
end_time = time.perf_counter()
print("Execution Time:", end_time - start_time, "seconds")
print("Time Complexity: O(1)")
print("Auxiliary Space Complexity: O(1)")
```

Task 03 — Array Traversal (Display all elements)

Traversal means visiting every element sequentially. It is used for printing, counting, summing, filtering, and processing all elements.

Solution:

```
import time
marks = [75, 82, 91, 68, 88, 79, 95, 84, 73, 90]
start_time = time.perf_counter()
for i in range(len(marks)):
    print("Index", i, ":", marks[i])


end_time = time.perf_counter()
print("Execution Time:", end_time - start_time, "seconds")
print("Time Complexity: O(n)")
print("Auxiliary Space Complexity: O(1)")
```

Task 04 — Calculate Average

Solution:

```
import time
marks = [75, 82, 91, 68, 88, 79, 95, 84, 73, 90]
start_time = time.perf_counter()
total = 0
for mark in marks:
    total += mark
average = total / len(marks)
end_time = time.perf_counter()
print("Average:", average)
print("Execution Time:", end_time - start_time, "seconds")
print("Time Complexity: O(n)")
print("Auxiliary Space Complexity: O(1)")
```

Complexity Analysis: Every mark is visited once. Therefore time is O(n) and auxiliary space is O(1).

Task 05 — Array Insertion (Insert at End)
Insertion adds an element at a position. Appending at the end is efficient, while inserting near the beginning or middle requires existing elements to shift.

Solution:

```
import time
students = ["Ali", "Sara", "Ahmed", "Hina"]
start_time = time.perf_counter()
students.append("Usman")
end_time = time.perf_counter()
print("Updated Array:", students)
print("Execution Time:", end_time - start_time, "seconds")
print("Time Complexity: O(1) amortized")
print("Auxiliary Space Complexity: O(1) amortized")
```

Complexity Analysis: append() is O(1) amortized. Python may occasionally resize the underlying list, but the amortized cost is constant.

Task 06 — Insert at Beginning

Solution:

```
import time
marks = [70, 75, 80, 85, 90]
start_time = time.perf_counter()
marks.insert(0, 65)
end_time = time.perf_counter()
```

```
print("Updated Array:", marks)
print("Execution Time:", end_time - start_time, "seconds")
print("Time Complexity: O(n)")
print("Auxiliary Space Complexity: O(1) auxiliary")
```

Complexity Analysis: Inserting at index 0 shifts existing elements to the right. The shifting grows with n, so time is O(n).

Task 07 — Array Deletion (Delete Last Element)
Deletion removes an element. Removing the last element requires no shifting, while deleting from the beginning or middle shifts subsequent elements left.

Solution:

```
import time
students = ["Ali", "Sara", "Ahmed", "Hina", "Usman"]
start_time = time.perf_counter()
removed = students.pop()
end_time = time.perf_counter()
print("Removed:", removed)
print("Updated Array:", students)
print("Execution Time:", end_time - start_time, "seconds")
print("Time Complexity: O(1) amortized")

print("Auxiliary Space Complexity: O(1)")
```

Complexity Analysis: pop() without an index removes the last element without shifting, so it is O(1) amortized.

Task 08 - (Delete First Element)

Solution:

```
import time
marks = [65, 70, 75, 80, 85, 90]
start_time = time.perf_counter()
removed = marks.pop(0)
end_time = time.perf_counter()
print("Removed:", removed)
print("Updated Array:", marks)
print("Execution Time:", end_time - start_time, "seconds")
print("Time Complexity: O(n)")
print("Auxiliary Space Complexity: O(1) auxiliary")
```

Complexity Analysis: Deleting index 0 shifts the remaining elements left, so the time complexity is O(n).

Exercise: Implement the following tasks, find out the Time Complexity and Space Complexityof each using pref.counter() method and classify the time complexity of each among O(1), O(n) and)(n^2):

Task-1: Create a list of 10 SAP IDs. Display the elements at index 0, index 5, and the last index. Measure execution time using time.perf_counter(). Explain why the accesses are O(1) and state the auxiliary space complexity.

Task-2: Create a list of marks for n students and count how many students obtained marks greater than or equal to 50. Test n = 100, 500, 1000, and 2000. Measure execution time and justify the time and auxiliary space complexity.

Task-3: Create a list of 10 student marks. Insert a new mark at the end, beginning, and a specified middle index. Measure each operation. Repeat with increasing input sizes and explain the complexity of each operation.

Task-4: Create a list of student names. Delete an element from the end, beginning, and a specified middle index. Measure execution time and explain how the number of shifted elements affects time complexity. State auxiliary space complexity.

Submission Requirements:

- Save the Python program/notebook using the format: SAPID_YourName_Lab01.
- Write your SAP ID and Name at the beginning of the notebook.
- Run all programs and include the output for each task.
- Add brief comments to explain important parts of the code.
- Submit the completed lab on the google classroom.

Aror University of Art, Architecture, Design & Heritage Sukkur.